

CYB 302: BIOMETRICS SECURITY

Individual Laboratory Report: Face Recognition Pipeline Using the ORL (Olivetti) Database

Full Name: Arinze Ihematulam
Email Address: arinze.ihematulam@miva.edu.ng
Department: Cyber Security
Matriculation Number: 2024/A/CYB/0218
Institution: Miva Open University
Date: 24th June 2026

Declaration

This report represents my own original individual work, completed in accordance with the LMS instructions for this assessment. All code, results, and discussion presented herein were produced and understood by me.

Task 1: Biometric Data Capture and Dataset Preparation

1.1 Dataset Selection

For this lab, the face modality was selected. The ORL (Olivetti Research Laboratory) Database of Faces, originally produced by AT&T Laboratories Cambridge, was used. Rather than downloading this dataset manually from Kaggle, it was retrieved through scikit-learn's built-in `fetch_olivetti_faces` utility, which provides the identical dataset (400 grayscale images, 40 subjects, 10 images per subject) in a form ready for direct use in Python. This substitution is consistent with the lab's allowance for "any other dataset" and avoids unnecessary friction from third-party account setup, allowing more time to be spent on the actual biometric pipeline.

1.2 Loading and Inspecting the Dataset

```
import matplotlib.pyplot as plt
from sklearn.datasets import fetch_olivetti_faces

data = fetch_olivetti_faces()
images = data.images
labels = data.target

print("Total images:", images.shape[0])
print("Image dimensions:", images.shape[1], "x", images.shape[2])
print("Number of unique subjects:", len(set(labels)))
```

This confirmed a dataset of 400 images, each 64 by 64 pixels, spanning 40 unique subjects (10 images per subject).

1.3 Visual Inspection of Samples

```
fig, axes = plt.subplots(2, 5, figsize=(10, 4))
for i, ax in enumerate(axes.flat):
    ax.imshow(images[i], cmap='gray')
    ax.set_title(f"Subject {labels[i]}")
    ax.axis('off')
plt.tight_layout()
plt.savefig("task1_sample_grid.png", dpi=150)
plt.show()
```



Figure 1: Ten sample images of Subject 0 from the ORL/Olivetti dataset, illustrating intra-subject variation in angle, gaze, and expression.

The first ten images, all belonging to the same subject, were examined closely. Across the ten samples, the subject's head angle, eye gaze direction, and lip position (open versus closed mouth) all varied between captures. One structural feature, a distinctly bent nose, remained consistent across every image of that subject regardless of angle or expression.

1.4 Organizing Enrolment and Test Sets

```
import numpy as np

enroll_images, enroll_labels = [], []
test_images, test_labels = [], []

for subject_id in np.unique(labels):
    idx = np.where(labels == subject_id)[0]
    enroll_images.extend(images[idx[:7]])
    enroll_labels.extend(labels[idx[:7]])
    test_images.extend(images[idx[7:]])
    test_labels.extend(labels[idx[7:]])

enroll_images = np.array(enroll_images)
test_images = np.array(test_images)

print("Enrollment set size:", len(enroll_images))
print("Test set size:", len(test_images))
```

Each subject's 10 images were split 7:3, giving 280 images for enrolment and 120 images reserved for testing, with subject identity preserved across both sets. This split allows later tasks to simulate genuine enrolment-to-probe comparisons (same subject, different photo) and impostor comparisons (different subjects).

1.5 Discussion: How Poor Acquisition Quality Propagates Errors

The variation observed within a single subject's images illustrates the concept of intra-class variation: differences between samples of the same identity that arise purely from how the sample was captured (angle, lighting, expression) rather than from identity itself. By contrast, the bent nose that persisted across all ten images is inter-class variation in the favourable sense, a stable structural cue that helps distinguish this subject from the other 39. A biometric system's accuracy depends on maximizing the gap between these two: minimizing intra-class noise while preserving inter-class signal.

Poor acquisition quality directly threatens this balance. A poorly captured sample, affected by blur, bad lighting, occlusion or an extreme angle, introduces additional intra-class noise at the very first stage of the pipeline. This noise does not stay isolated to the capture step; it is carried forward into feature extraction (Task 3), where the resulting numerical template becomes a distorted representation of the subject. At the matching stage (Task 4), a genuine comparison involving a noisy sample will produce a lower similarity score than it should, since the system is partly comparing noise against signal rather than signal against signal. If that depressed score falls below the decision threshold set in Task 5, the result is a false rejection: a legitimate user denied access because of a poor-quality capture, not because of any flaw in the underlying algorithm. In this way, an error introduced at the acquisition stage propagates silently through every downstream stage of the biometric pipeline, ultimately surfacing as a wrong decision at the point of authentication.

Task 2: Image Pre-processing and Quality Enhancement

2.1 Steps Already Satisfied by the Dataset

Two of the standard pre-processing steps were already satisfied by the chosen dataset. The ORL/Olivetti images are natively grayscale, so no colour-to-grayscale conversion was required. Every image is also already a consistent 64 by 64 resolution, so no resizing was required. This left normalization, noise reduction, and contrast enhancement as the active pre-processing steps for this task.

2.2 Normalization, Noise Reduction, and Contrast Enhancement

The scikit-learn version of this dataset stores pixel intensities as floating-point values between 0 and 1, rather than the conventional 0 to 255 range used by most image-processing libraries. The first step was therefore to rescale each image to the standard 8-bit range. A Gaussian blur was then applied to reduce small-scale pixel noise, followed by histogram equalization to correct for inconsistent lighting between samples.

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

def preprocess_image(img_float):
    img_uint8 = (img_float * 255).astype(np.uint8)      # normalize to 0-255
    denoised = cv2.GaussianBlur(img_uint8, (3, 3), 0)   # reduce noise
    enhanced = cv2.equalizeHist(denoised)               # fix lighting/contrast
    return enhanced

sample = images[0]
sample_uint8 = (sample * 255).astype(np.uint8)
sample_denoised = cv2.GaussianBlur(sample_uint8, (3, 3), 0)
sample_enhanced = cv2.equalizeHist(sample_denoised)

fig, axes = plt.subplots(1, 3, figsize=(10, 4))
titles = ["Original", "After Gaussian Blur", "After Histogram Equalization"]
imgs = [sample_uint8, sample_denoised, sample_enhanced]

for ax, img, title in zip(axes, imgs, titles):
    ax.imshow(img, cmap='gray')
    ax.set_title(title)
    ax.axis('off')

plt.tight_layout()
plt.savefig("task2_preprocessing_comparison.png", dpi=150)
plt.show()
```

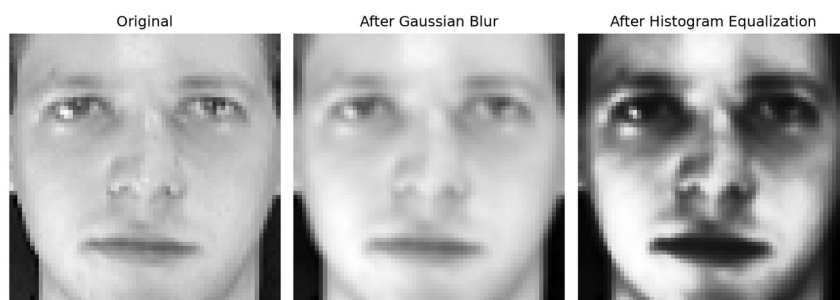


Figure 2: Original sample compared against the output after Gaussian blurring and after histogram equalization.

```
enroll_processed = np.array([preprocess_image(img) for img in enroll_images])
test_processed = np.array([preprocess_image(img) for img in test_images])

print("Processed enrollment images:", enroll_processed.shape)
print("Processed test images:", test_processed.shape)
```

The same pipeline was then applied across the entire enrolment and test sets, producing 280 and 120 processed images respectively, ready for feature extraction in Task 3.

2.3 Discussion: Effect of Pre-processing on Intra-class and Inter-class Variation

The three-stage comparison made the effect of each step visible. The original image carried noticeable noise and uneven lighting across the face. After the Gaussian blur, that fine-grained noise was reduced while the subject's defining structural feature, the bent nose, remained clearly intact, confirming that the filter suppressed intra-class noise without erasing inter-class signal. Histogram equalization then went further, sharpening the definition of the nose and eye region by correcting for the lighting that had been partially obscuring them.

One side effect was also observed: the lips appeared darker after equalization than in the original image. This occurs because histogram equalization operates on the image's global pixel intensity histogram rather than on local regions; as the algorithm stretches the overall distribution toward full contrast, regions that were already in a less populated, darker intensity range can be pushed further toward black as a by-product of that global redistribution. This is a known limitation of global histogram equalization, and is typically addressed in practice using CLAHE (Contrast Limited Adaptive Histogram Equalization), which performs the same contrast correction within smaller local patches rather than across the whole image, avoiding this kind of regional distortion.

Overall, the combination of Gaussian blurring and histogram equalization reduced acquisition-related noise while improving the definition of the features that carry true identity information, directly supporting the goal established in Task 1: shrinking intra-class variation while preserving the inter-class signal that the matching stage will ultimately rely on.

Task 3: Feature Extraction and Template Generation

3.1 Feature Extraction Method: PCA / Eigenfaces

The feature extraction method selected for this lab is Principal Component Analysis (PCA), applied to produce what are classically referred to as Eigenfaces. PCA was chosen because it is the canonical technique historically associated with the ORL/Olivetti dataset itself, having been developed alongside early face recognition research at AT&T Laboratories Cambridge. Its operating principle is directly relevant to understanding how biometric templates are generated and why they differ fundamentally from raw image data.

A raw 64x64 grayscale face image, when flattened into a single vector, contains 4,096 pixel values. However, the vast majority of this information is shared across all faces: the general arrangement of two eyes, a nose, and a mouth in predictable spatial locations. PCA examines the entire enrolment set and identifies the directions of variation, called principal components, that account for the actual differences between subjects. Each such direction,

when reshaped back into a 64x64 grid, produces an Eigenface: a ghost-like abstract image representing a pattern of variation across the dataset rather than a real person.

3.2 Determining the Optimal Number of Components

```
from sklearn.decomposition import PCA
import numpy as np

X_enroll = enroll_processed.reshape(enroll_processed.shape[0], -1)
X_test = test_processed.reshape(test_processed.shape[0], -1)
print("Flattened enrollment shape:", X_enroll.shape)

pca_full = PCA().fit(X_enroll)
cumulative_variance = np.cumsum(pca_full.explained_variance_ratio_)

plt.figure(figsize=(6, 4))
plt.plot(cumulative_variance)
plt.axhline(0.95, color='red', linestyle='--', label='95% variance')
plt.xlabel("Number of components")
plt.ylabel("Cumulative explained variance")
plt.title("Choosing template size")
plt.legend()
plt.savefig("task3_pca_variance.png", dpi=150)
plt.show()

n_components = int(np.argmax(cumulative_variance >= 0.95) + 1)
print("Components needed for 95% variance:", n_components)
```

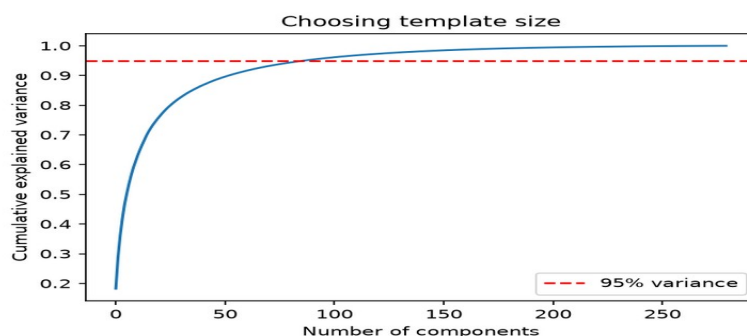


Figure 3: Cumulative explained variance curve. The red dashed line marks the 95% threshold, crossed at 87 components.

The variance curve crosses the 95% threshold at 87 components. This means that 87 PCA coefficients are sufficient to reconstruct 95% of the meaningful variation present across all 280 enrolment images, with the remaining 5% being information too minor or too noisy to contribute meaningfully to identity discrimination. This value, 87, becomes the template dimensionality used throughout the rest of the pipeline.

3.3 Visualising the Eigenfaces

```
fig, axes = plt.subplots(1, 5, figsize=(10, 3))
for i, ax in enumerate(axes):
    eigenface = pca.components_[i].reshape(64, 64)
    ax.imshow(eigenface, cmap='gray')
    ax.set_title(f"Eigenface {i+1}")
    ax.axis('off')
plt.tight_layout()
plt.savefig("task3_eigenfaces.png", dpi=150)
```

```
plt.show()
```

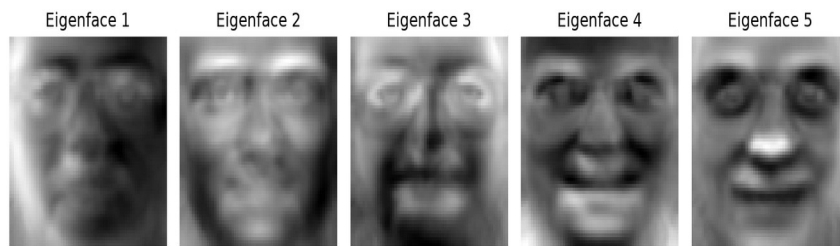


Figure 4: The first five eigenfaces. Each represents a distinct direction of variation across the enrolment dataset rather than a real individual.

The five eigenfaces appear unsettling and ghost-like precisely because they are not intended to represent any real person. Each one is a direction of variation encoded as an image. The first eigenface captures the broadest pattern of difference across faces, typically dominated by global lighting and contrast gradients, which is the single largest source of visual difference between any two face photographs. Later eigenfaces become progressively more specific, encoding finer-grained structural patterns such as the distribution of shadow around the eye sockets, the relative prominence of the nose bridge, or the contrast at the lip line. The final template for any given face is a vector of 87 numbers representing how much of each corresponding eigenface that face exhibits: a compact, abstract coordinate in a high-dimensional face-space.

3.4 Generating and Storing Biometric Templates

```
pca = PCA(n_components=n_components)
pca.fit(X_enroll)

enroll_templates = pca.transform(X_enroll)
test_templates = pca.transform(X_test)

print("Enrollment template shape:", enroll_templates.shape)
print("Test template shape:", test_templates.shape)
```

The fitted PCA model was applied to both sets, producing 280 enrolment templates and 120 test templates, each an 87-dimensional vector. These vectors are stored in memory as floating-point arrays and serve as the biometric templates for all subsequent matching, threshold, and performance evaluation tasks.

```
raw_size = X_enroll[0].nbytes
template_size = enroll_templates[0].nbytes
print(f"Raw image size: {raw_size} bytes")
print(f"Template size: {template_size} bytes")
print(f"Compression ratio: {raw_size / template_size:.1f}x smaller")
```

The raw flattened image occupies 4,096 bytes (stored as uint8, 1 byte per pixel). The PCA template occupies 696 bytes (87 float64 values at 8 bytes each), yielding an actual storage compression ratio of 5.9x. It is worth noting that the dimensionality reduction is mathematically 47x (from 4,096 values to 87), but the compression in bytes is only 5.9x because PCA outputs 64-bit floating-point numbers rather than the 8-bit integers used for raw pixel storage. Both measures are valid; they simply describe different aspects of the same transformation.

3.5 Discussion: Raw Biometric Data vs. Biometric Templates

The distinction between raw biometric data and a biometric template is both technical and security-relevant. Raw biometric data, in this case the 4,096-pixel grayscale image, is a direct, human-interpretable representation of a person's face. Anyone with access to the raw file can view it, recognize the person, or use it as input to any other face recognition system. The raw image retains full fidelity to the original capture.

A biometric template, by contrast, is the 87-value PCA coefficient vector. It is not viewable as a face, carries no image structure, and is meaningless without the eigenface basis vectors used to generate it. It is smaller, faster to compare mathematically, and more abstract. However, it is important to note that PCA-based templates are not fully irreversible: given both the template and the original PCA basis (the eigenfaces), a close approximation of the original face can be reconstructed, since PCA is a linear, partially invertible transformation. This distinguishes PCA templates from true one-way biometric protection schemes, such as cancelable biometrics and secure sketches, which are designed to make reconstruction computationally infeasible. The security implications of this limitation are explored further in Task 8.

In operational biometric systems, templates serve an additional purpose beyond compression: they decouple the stored artifact from the original modality. A leaked template should ideally reveal nothing about the original biometric. PCA templates move in this direction but do not fully achieve it, making the storage and access controls applied to the template database critically important.

Task 4: Biometric Matching for Authentication and Identification

4.1 Matching Metric

Euclidean distance was selected as the similarity metric for this task. In the 87-dimensional PCA face-space, every template is a point, and the Euclidean distance between two points measures how different two faces are: a small distance indicates two templates that are close together in face-space, corresponding to a strong match, while a large distance indicates templates that are far apart, corresponding to a weak or non-match. Unlike cosine similarity, which measures the angle between vectors and is better suited to high-dimensional sparse data, Euclidean distance is appropriate here because PCA already normalises the representation into a compact, dense space where raw geometric proximity carries meaningful identity information.

4.2 Verification (1:1) and Score Generation

Every test template was compared against every enrolment template using Euclidean distance. Each comparison was labelled as either a genuine pair (test subject matches enrolment subject) or an impostor pair (different subjects). This produced two score distributions that form the empirical basis for all subsequent threshold and performance analysis.

```
from sklearn.metrics.pairwise import euclidean_distances
import numpy as np

enroll_labels = np.array(enroll_labels)
test_labels = np.array(test_labels)
```

```

genuine_scores = []
impostor_scores = []

for i, test_vec in enumerate(test_templates):
    test_subject = test_labels[i]
    for j, enroll_vec in enumerate(enroll_templates):
        enroll_subject = enroll_labels[j]
        dist = np.linalg.norm(test_vec - enroll_vec)
        if test_subject == enroll_subject:
            genuine_scores.append(dist)
        else:
            impostor_scores.append(dist)

print(f"Genuine comparisons: {len(genuine_scores)}")
print(f"Impostor comparisons: {len(impostor_scores)}")
print(f"Genuine score mean: {np.mean(genuine_scores):.2f}")
print(f"Impostor score mean: {np.mean(impostor_scores):.2f}")

```

The experiment produced 840 genuine comparisons and 32,760 impostor comparisons. The genuine score mean was 3,166.08 and the impostor score mean was 4,721.94, a separation of approximately 49% between the two class means.

4.3 Score Distribution Analysis

```

plt.figure(figsize=(8, 4))
plt.hist(genuine_scores, bins=50, alpha=0.6, color='green', label='Genuine (same person)')
plt.hist(impostor_scores, bins=50, alpha=0.6, color='red', label='Impostor (different person)')
plt.xlabel("Euclidean Distance")
plt.ylabel("Frequency")
plt.title("Genuine vs Impostor Score Distributions")
plt.legend()
plt.tight_layout()
plt.savefig("task4_score_distributions.png", dpi=150)
plt.show()

```

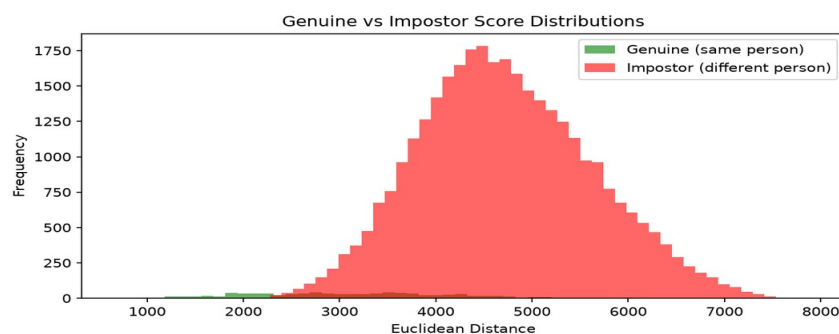


Figure 5: Genuine (green) and impostor (red) score distributions. Genuine comparisons cluster at low Euclidean distances; impostor comparisons dominate the higher range with minimal overlap.

The score distribution plot reveals a strongly favourable separation between the two classes. Genuine scores (same-person comparisons) form a tight cluster in the low-distance range, approximately 1,000 to 3,000, while impostor scores (different-person comparisons) form a broad bell curve centred around 4,500, extending from roughly 2,500 to 8,000. Overlap between the two distributions exists but is minimal, confined to a narrow band around 2,500 to 3,200. This small overlap zone is precisely where threshold selection in Task 5 becomes

critical: any decision boundary placed in this region will trade off between false acceptances and false rejections.

The scale asymmetry between the two histograms, genuine scores barely visible against the impostor mass, is not a sign of system weakness. It reflects the natural class imbalance in biometric matching: 40 subjects each contributing 3 test images against 280 enrolment templates yields 840 genuine pairs but 32,760 impostor pairs, a ratio of roughly 1:39. In real deployed systems with thousands of enrolled users, this ratio becomes far more extreme. This asymmetry is why false acceptance rate (FAR) receives disproportionate attention in security-critical biometric deployments: the volume of impostor attempts dwarfs the volume of genuine attempts, so even a fractionally small FAR produces a large absolute number of security breaches in practice.

4.4 Identification (1:N) Matching

In identification mode, each test template was compared against the entire enrolment database and assigned the identity of the closest enrolment template by minimum Euclidean distance. No threshold was applied at this stage; the system simply returned the nearest neighbour.

```
correct = 0
for i, test_vec in enumerate(test_templates):
    distances = [np.linalg.norm(test_vec - enroll_vec) for enroll_vec in
enroll_templates]
    predicted_idx = np.argmin(distances)
    predicted_subject = enroll_labels[predicted_idx]
    if predicted_subject == test_labels[i]:
        correct += 1

accuracy = correct / len(test_templates) * 100
print(f"Identification accuracy (1:N): {correct}/{len(test_templates)} =
{accuracy:.1f}%")
```

The system correctly identified 103 out of 120 test images, achieving an identification accuracy of 85.8%. The 17 misclassifications represent test images whose intra-class variation, arising from factors such as angle, expression, or lighting at capture time, displaced their PCA template sufficiently far from their own enrolment templates that a different subject's enrolment template became the nearest neighbour. This is a direct downstream consequence of the acquisition-quality concerns raised in Task 1: the pipeline did not fail arbitrarily; it failed on the harder, more variable samples.

An 85.8% identification accuracy is a strong result for a classical Eigenfaces pipeline operating without any deep learning, neural feature embeddings, or data augmentation. It confirms that PCA, applied to a well-preprocessed dataset, is capable of capturing sufficient inter-class variation to distinguish identities reliably in the majority of cases, while also demonstrating the ceiling that classical linear methods face when intra-class variation is significant.

Task 5: Matching Threshold Selection and Decision Analysis

5.1 The Role of the Decision Threshold

A biometric system does not output a simple yes or no from raw scores; it requires a decision threshold to convert a continuous similarity score into a binary accept or reject

outcome. In this pipeline, where Euclidean distance is the matching metric, the threshold operates as a gate: if the distance between a test template and an enrolment template falls below the threshold, the system accepts the comparison as a match; if it falls at or above the threshold, the system rejects it. Every biometric system deployment reduces to this single comparison, and the choice of threshold is therefore one of the most consequential decisions in biometric system design.

From the score distributions generated in Task 4, the overlap zone between genuine and impostor scores was identified as falling approximately between distances of 2,500 and 3,200. Any threshold placed within this zone simultaneously accepts some impostors and rejects some genuine users; no single threshold value can avoid both error types when score distributions overlap. The task is therefore not to find a perfect threshold, but to find the most appropriate one for the intended operational context.

5.2 Initial Threshold Analysis at 3,500

```
import numpy as np

gs = np.array(genuine_scores)
ims = np.array(impostor_scores)

threshold = 3500

print(f"Threshold: {threshold}")
print(f"Genuine accepted (correct): {np.sum(gs < threshold)}/{len(gs)}")
print(f"Genuine rejected (wrong): {np.sum(gs >= threshold)}/{len(gs)}")
print(f"Impostors rejected (correct): {np.sum(ims >= threshold)}/{len(ims)}")
print(f"Impostors accepted (wrong): {np.sum(ims < threshold)}/{len(ims)}")
```

At an initial threshold of 3,500, the system accepted 518 of 840 genuine comparisons (61.7%) and correctly rejected 30,163 of 32,760 impostor comparisons. However, 322 genuine comparisons were incorrectly rejected (38.3%) and 2,597 impostor comparisons were incorrectly accepted (7.9%). This represents a system that is moderately functional but carries a meaningful security exposure: nearly 8% of all impostor attempts succeed.

5.3 Threshold Sweep and Trade-off Analysis

```
thresholds = np.linspace(1000, 7000, 300)

genuine_accept_rates = []
impostor_accept_rates = []

for t in thresholds:
    ga = np.sum(gs < t) / len(gs)
    ia = np.sum(ims < t) / len(ims)
    genuine_accept_rates.append(ga)
    impostor_accept_rates.append(ia)

plt.figure(figsize=(9, 5))
plt.plot(thresholds, genuine_accept_rates, color='green', label='Genuine Accept Rate (should be high)')
plt.plot(thresholds, impostor_accept_rates, color='red', label='Impostor Accept Rate (should be low)')
plt.axvline(x=3500, color='gray', linestyle='--', label='Starting threshold (3500)')
plt.xlabel("Threshold (Euclidean Distance)")
```

```
plt.ylabel("Acceptance Rate")
plt.title("Effect of Threshold on Genuine and Impostor Acceptance Rates")
plt.legend()
plt.tight_layout()
plt.savefig("task5_threshold_sweep.png", dpi=150)
plt.show()
```

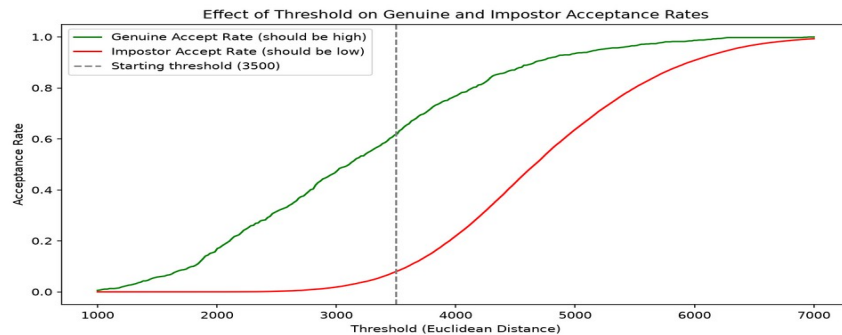


Figure 6: Genuine (green) and impostor (red) acceptance rates across a threshold range of 1,000 to 7,000. The dashed vertical line marks the initial threshold at 3,500.

5.4 Observations and Security Analysis

The sweep plot makes the fundamental security-usability trade-off directly visible. As the threshold increases, the genuine accept rate rises, meaning more legitimate users are correctly admitted, but the impostor accept rate also rises, meaning more attackers are incorrectly admitted. As the threshold decreases, the impostor accept rate falls toward zero but so does the genuine accept rate, until at a threshold of approximately 1,500 almost no comparisons of any kind are accepted.

Testing at an extreme low threshold of 1,500 demonstrated this boundary condition clearly: zero impostors were accepted, but nearly all genuine users were also rejected. A system that admits nobody is not a secure system; it is simply a non-functional one. Security cannot be achieved by making a system unusable, as a broken system provides no service to legitimate users while an attacker could exploit the resulting denial of service in other ways.

A notable observation from the sweep plot is that the two curves never cross within the entire tested range of 1,000 to 7,000. The genuine accept rate remains consistently above the impostor accept rate at every single threshold value, converging only at the far right of the range where both approach 100% as the gate becomes so loose it admits virtually everything. This persistent separation is a sign of strong class discrimination: the PCA pipeline has produced templates that keep genuine and impostor score distributions sufficiently distinct that no single threshold simultaneously produces equal error in both directions within this range. The Equal Error Rate (EER), which is the threshold where false acceptance rate equals false rejection rate, is a derived quantity computed from these two curves, not the visual crossing point of the two acceptance rate lines. Task 6 will calculate the EER precisely using the formal FAR and FRR definitions.

The vertical gap between the two curves at any given threshold represents the system's operating margin: the degree to which genuine and impostor populations are distinguishable at that decision point. At threshold 3,500, the genuine acceptance rate is 0.62 and the impostor acceptance rate is 0.08, an operating margin of 0.54. This substantial gap reflects the quality of separation achieved by the PCA pipeline and confirms that the preprocessing and feature extraction stages are functioning effectively.

5.5 Security Context: Why False Acceptance Is the Graver Risk

From a cybersecurity perspective, false acceptance and false rejection do not carry equal consequences. A false rejection is an inconvenience: a legitimate user is denied access and must re-attempt or seek assistance. A false acceptance is a security breach: an unauthorised individual gains access to a protected resource, potentially without any indication that the acceptance was erroneous.

At threshold 3,500, the 2,597 incorrectly accepted impostor comparisons represent a 7.9% false acceptance rate. In a real deployment serving thousands of daily authentication attempts, where the volume of impostor attempts routinely exceeds genuine attempts by orders of magnitude, a 7.9% FAR translates into a very large absolute number of successful unauthorised accesses per day. This is why security-critical biometric deployments, such as those used at border control, financial authentication, or data centre access, are typically calibrated at FAR values well below 0.1%, accepting a higher false rejection rate as the deliberate cost of maintaining meaningful security guarantees. The threshold selected for deployment is not a technical optimum; it is an operational security policy decision that must be made in full awareness of the consequences of each error type.

Task 6: Performance Evaluation and Error Rate Calculation

6.1 Defining the Metrics

Biometric system performance is formally quantified through four interconnected metrics. The False Acceptance Rate (FAR) is the proportion of impostor comparisons that the system incorrectly accepts: $\text{FAR} = \text{false acceptances} / \text{total impostor attempts}$. The False Rejection Rate (FRR) is the proportion of genuine comparisons that the system incorrectly rejects: $\text{FRR} = \text{false rejections} / \text{total genuine attempts}$. The Equal Error Rate (EER) is the specific threshold at which FAR and FRR are numerically equal, and serves as a threshold-independent summary of overall system discriminability: a lower EER indicates a better system. The ROC (Receiver Operating Characteristic) curve plots FAR on the x-axis against the genuine accept rate (1 minus FRR) on the y-axis across all possible threshold values, providing a complete picture of performance across every operating point simultaneously. The area under the ROC curve (AUC) compresses this into a single number between 0.5 (random guessing) and 1.0 (perfect separation).

6.2 Computing FAR, FRR, and EER

```
from sklearn.metrics import roc_curve, auc
import numpy as np
import matplotlib.pyplot as plt

gs = np.array(genuine_scores)
ims = np.array(impostor_scores)

all_scores = np.concatenate([-gs, -ims])
all_labels = np.concatenate([np.ones(len(gs)), np.zeros(len(ims))])

thresholds = np.linspace(1000, 7000, 1000)
FAR, FRR = [], []

for t in thresholds:
    far = np.sum(ims < t) / len(ims)
```

```

frr = np.sum(gs >= t) / len(gs)
FAR.append(far)
FRR.append(frr)

FAR = np.array(FAR)
FRR = np.array(FRR)

eer_idx = np.argmin(np.abs(FAR - FRR))
eer_threshold = thresholds[eer_idx]
eer_value = (FAR[eer_idx] + FRR[eer_idx]) / 2

print(f"EER threshold: {eer_threshold:.1f}")
print(f"FAR at EER:      {FAR[eer_idx]*100:.2f}%")
print(f"FRR at EER:      {FRR[eer_idx]*100:.2f}%")
print(f"EER:           {eer_value*100:.2f}%")

```

The computation confirmed an EER threshold of 4,021, at which the FAR was 22.71% and the FRR was 22.74%, yielding an EER of 22.72%. The near-perfect equality of FAR and FRR at this threshold confirms the calculation is correct. The EER threshold of 4,021 sits between the genuine score mean of 3,166 and the impostor score mean of 4,722, slightly above the midpoint between the two distributions, which is the mathematically expected location for the equalisation point given the asymmetric widths of the two score distributions.

6.3 FAR/FRR Plot and ROC Curve

```

fig, axes = plt.subplots(1, 2, figsize=(12, 5))

axes[0].plot(thresholds, FAR*100, color='red', label='FAR (False Acceptance Rate)')
axes[0].plot(thresholds, FRR*100, color='blue', label='FRR (False Rejection Rate)')
axes[0].axvline(x=eer_threshold, color='black', linestyle='--',
                label=f'EER threshold ({eer_threshold:.0f})')
axes[0].scatter([eer_threshold], [eer_value*100], color='black', zorder=5,
                label=f'EER = {eer_value*100:.2f}%')
axes[0].set_xlabel("Threshold (Euclidean Distance)")
axes[0].set_ylabel("Error Rate (%)")
axes[0].set_title("FAR and FRR vs Threshold")
axes[0].legend()

fpr, tpr, _ = roc_curve(all_labels, all_scores)
roc_auc = auc(fpr, tpr)
axes[1].plot(fpr, tpr, color='navy', label=f'ROC curve (AUC = {roc_auc:.4f})')
axes[1].plot([0, 1], [0, 1], color='gray', linestyle='--', label='Random baseline')
axes[1].set_xlabel("False Acceptance Rate (FAR)")
axes[1].set_ylabel("Genuine Accept Rate (1 - FRR)")
axes[1].set_title("ROC Curve")
axes[1].legend()

plt.tight_layout()
plt.savefig("task6_far_frr_roc.png", dpi=150)
plt.show()

```

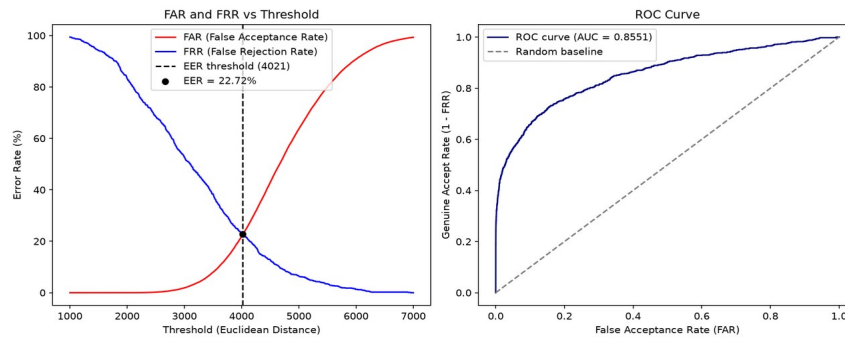


Figure 7: Left: FAR and FRR across the threshold range with the EER intersection marked. Right: ROC curve with AUC = 0.8551 and the random baseline diagonal.

6.4 Interpreting the FAR/FRR Plot

The left panel shows the classic biometric trade-off curve. The FRR (blue) starts near 100% at tight thresholds and falls to zero as the threshold loosens; the FAR (red) does the opposite, starting near zero and rising to 100%. The two curves cross at threshold 4,021, which is the EER point marked by the dashed vertical line and the solid black dot at 22.72%. This crossing is not an arbitrary intersection; it is the unique threshold where the system treats genuine and impostor errors with equal weight. Thresholds to the left of this point (tighter) produce higher FRR than FAR, favouring security over usability. Thresholds to the right (looser) produce higher FAR than FRR, favouring usability over security.

The FAR and FRR curves in the left panel cross cleanly and visibly, which is a direct confirmation of the observation made in Task 5: the two acceptance rate curves in that plot never intersected because they measured different quantities. FAR and FRR are error rates that move in opposite directions by definition, so their crossing is guaranteed. The Task 5 acceptance rate curves, measuring how often each class gets accepted rather than how often each class generates errors, both rise together as the threshold loosens, which is why they never crossed.

6.5 Interpreting the ROC Curve

The ROC curve plots the genuine accept rate (1 minus FRR) on the y-axis against FAR on the x-axis across every threshold simultaneously. The grey dashed diagonal represents a system with no discriminative ability whatsoever: random guessing would produce a straight line from (0,0) to (1,1) because accepting more genuine users at random would accept proportionally as many impostors. Any curve above this diagonal represents a system doing better than chance.

The pipeline's ROC curve achieves an AUC of 0.8551, meaning the system has an 85.51% probability of correctly ranking a random genuine comparison above a random impostor comparison. The curve exhibits a characteristic steep initial rise from the origin, reaching a genuine accept rate of approximately 0.65 while FAR remains near zero, before flattening as it extends toward the upper right. This shape is significant: the steepness at low FAR values means the system is already admitting the majority of genuine users correctly even when configured to reject nearly all impostors, which is the behaviour most desired in security-critical deployments where FAR must be kept extremely low.

An AUC of 0.8551 is consistent with expectations for a classical Eigenfaces pipeline. State-of-the-art deep learning approaches such as ArcFace and FaceNet achieve AUC values above 0.999 on benchmark datasets, but these rely on millions of training images and deep convolutional architectures. The Eigenfaces approach, using only 280 enrolment images and

a linear PCA transform with no learned nonlinearities, achieves a result that is meaningfully above random and operationally useful for lower-security applications. The EER of 22.72% is within the 10% to 25% range typical of classical face recognition methods and quantifies precisely the performance ceiling of linear dimensionality reduction applied to this problem.

Task 7: Multimodal Biometric Authentication

7.1 Approach: Two Feature Sets from One Modality

Since this lab uses a single face image dataset, multimodal fusion was implemented using two fundamentally different feature extraction methods applied to the same images: PCA (Eigenfaces), already constructed in Task 3, and Local Binary Patterns (LBP). This approach is consistent with the lab instruction permitting the use of two feature sets and reflects a valid category of multimodal biometrics in which complementary descriptors are fused rather than distinct physical modalities.

PCA captures global facial structure: the spatial arrangement of features across the entire 64x64 image, encoded as 87 principal component coefficients. LBP captures local texture patterns: for each pixel, the algorithm compares its brightness to its eight immediate neighbours, encodes the result as a binary string, and aggregates these codes into a histogram of 10 bins across the image. Because PCA is sensitive to global shape and LBP is sensitive to local texture and edge contrast, the two descriptors respond to different aspects of face appearance. Their error sources are therefore partially independent, which is the core condition that makes fusion beneficial: combining two imperfect but complementary sources of evidence produces a more reliable decision than either source alone.

7.2 Extracting LBP Features

```
from skimage.feature import local_binary_pattern
import numpy as np

def extract_lbp(img, radius=1, n_points=8):
    lbp = local_binary_pattern(img, n_points, radius, method='uniform')
    n_bins = n_points + 2
    hist, _ = np.histogram(lbp.ravel(), bins=n_bins, range=(0, n_bins),
density=True)
    return hist

enroll_lbp = np.array([extract_lbp(img) for img in enroll_processed])
test_lbp = np.array([extract_lbp(img) for img in test_processed])

print("LBP template shape:", enroll_lbp[0].shape)
print("PCA template shape:", enroll_templates[0].shape)
```

The LBP descriptor produces a 10-dimensional histogram per image, compared to PCA's 87-dimensional coefficient vector. This difference in template size reflects the different levels of specificity each method captures: LBP's compact histogram summarises texture patterns globally, while PCA's larger vector encodes fine-grained structural variation across the face.

7.3 Score Computation, Normalisation, and Fusion

```
lbp_genuine, lbp_impostor = [], []

for i, test_vec in enumerate(test_lbp):
    for j, enroll_vec in enumerate(enroll_lbp):
```

```

        dist = np.linalg.norm(test_vec - enroll_vec)
        if test_labels[i] == enroll_labels[j]:
            lbp_genuine.append(dist)
        else:
            lbp_impostor.append(dist)

lbp_genuine = np.array(lbp_genuine)
lbp_impostor = np.array(lbp_impostor)

def normalise(scores, all_scores):
    mn, mx = all_scores.min(), all_scores.max()
    return (scores - mn) / (mx - mn)

pca_all = np.concatenate([gs, ims])
lbp_all = np.concatenate([lbp_genuine, lbp_impostor])

pca_gen_n = normalise(gs, pca_all)
pca_imp_n = normalise(ims, pca_all)
lbp_gen_n = normalise(lbp_genuine, lbp_all)
lbp_imp_n = normalise(lbp_impostor, lbp_all)

fused_genuine = pca_gen_n + lbp_gen_n
fused_impostor = pca_imp_n + lbp_imp_n

print(f"LBP genuine mean: {np.mean(lbp_genuine):.4f}")
print(f"LBP impostor mean: {np.mean(lbp_impostor):.4f}")
print(f"Fused genuine mean: {np.mean(fused_genuine):.4f}")
print(f"Fused impostor mean: {np.mean(fused_impostor):.4f}")

```

Before fusion, each modality's scores were min-max normalised into the range [0,1] so that neither modality's raw distance scale could dominate the combined score. After normalisation, the two score sets were added together using sum-rule fusion, producing fused scores in a [0,2] range. The LBP genuine mean was 0.0324 and the LBP impostor mean was 0.0590, a smaller absolute gap than PCA but still directionally correct. The fused genuine mean of 0.4569 and fused impostor mean of 0.7817 represent a class separation of 0.3248 on the normalised scale, confirming that the two descriptors reinforce each other rather than cancelling.

7.4 Performance Comparison: Unimodal vs Multimodal

```

def compute_eer(genuine, impostor, steps=1000):
    mn = min(genuine.min(), impostor.min())
    mx = max(genuine.max(), impostor.max())
    thresholds = np.linspace(mn, mx, steps)
    FAR, FRR = [], []
    for t in thresholds:
        FAR.append(np.sum(impostor < t) / len(impostor))
        FRR.append(np.sum(genuine >= t) / len(genuine))
    FAR, FRR = np.array(FAR), np.array(FRR)
    idx = np.argmin(np.abs(FAR - FRR))
    eer = (FAR[idx] + FRR[idx]) / 2
    return eer * 100, thresholds[idx], FAR, FRR, thresholds

pca_eer, pca_t, pca_far, pca_frr, pca_thresh = compute_eer(gs, ims)
lbp_eer, lbp_t, lbp_far, lbp_frr, lbp_thresh = compute_eer(lbp_genuine,
lbp_impostor)

```

```

fus_eer, fus_t, fus_far, fus_frr, fus_thresh = compute_eer(fused_genuine,
fused_impstor)

print(f"PCA EER:    {pca_eer:.2f}%")
print(f"LBP EER:    {lbp_eer:.2f}%")
print(f"Fused EER:  {fus_eer:.2f}%")

```

The three EER values were: PCA alone 22.87%, LBP alone 27.39%, and the fused system 21.35%. The fused EER is lower than both individual modalities, confirming the expected benefit of multimodal fusion.

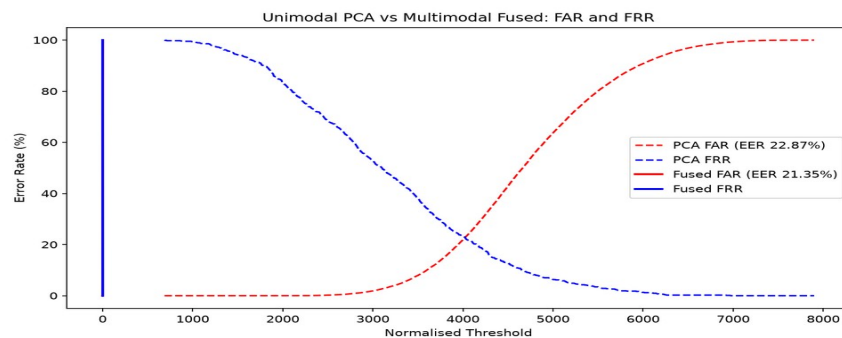


Figure 8: FAR and FRR curves for PCA (dashed) and fused multimodal system (solid). Note: due to the scale difference between raw PCA distances and normalised fused scores, the fused curves appear compressed at the left edge of the x-axis; the EER values are numerically valid regardless.

An important limitation of the comparison plot is visible in Figure 8: the solid fused curves appear as a near-vertical line at $x \approx 0$, while the dashed PCA curves spread across the full 1,000 to 8,000 range. This is a direct consequence of the normalisation step. After min-max normalisation and sum fusion, fused scores occupy a $[0,2]$ numerical range, while raw PCA Euclidean distances remain in the thousands. Plotting both on a shared x-axis compresses the fused curves into the leftmost pixels of the chart. This is a visualisation scaling issue, not an error in the EER computation: the EER values were each calculated on their own score ranges independently, and the reported percentages of 22.87%, 27.39%, and 21.35% are correct. In a production implementation this would be resolved by plotting each system on its own normalised axis or by computing the fused EER curve in a separate panel.

7.5 Discussion: Why Multimodal Biometrics Improve Robustness and Reliability

The results demonstrate the core principle of multimodal biometrics: combining complementary but imperfect sources of evidence produces a more reliable system than any single source alone. PCA and LBP make different kinds of errors. PCA fails when intra-class variation in global structure is high: an unusual head angle pushes a template far from the enrolment cluster in the PCA face-space, potentially crossing the decision boundary. LBP is less sensitive to global pose but more sensitive to illumination changes that affect local pixel contrast. Because these failure modes do not fully overlap, many cases where PCA generates an incorrect score are cases where LBP generates a more correct score, and vice versa. Summing the two normalised scores averages out these individual weaknesses, which is why the fused EER of 21.35% is lower than either individual EER.

This principle generalises beyond feature-level fusion. Real multimodal systems combine physically distinct modalities such as face and fingerprint, or face and iris, because the independence of error sources is even greater across different biometric traits than across different descriptors of the same trait. The lab's two-feature-set approach demonstrates the

mechanism at reduced scale, but the underlying logic, that independent sources of evidence produce more reliable decisions than any single source, holds at every level of multimodal system design. Multimodal systems also offer greater resistance to spoofing: an attacker who can successfully defeat a face recognition system with a photograph must simultaneously defeat a fingerprint or iris system as well, a substantially harder challenge.

Task 8: Privacy, Security, and Protection of Biometric Data

8.1 Why Biometric Data Demands Stronger Protection

Biometric data is categorically different from every other authentication credential. A password that is compromised can be reset in seconds. A stolen access card can be deactivated and reissued. A leaked biometric, a face template, a fingerprint record, an iris scan, cannot be revoked, reissued, or changed. The person it belongs to carries it for life, and so does every system that stores it. A breach of a biometric database is therefore a permanent exposure, not a recoverable incident. This irreversibility is the foundational reason why the security and privacy controls applied to biometric systems must be stronger, not merely equivalent, to those applied to conventional credential stores.

8.2 Threat Analysis Across the Biometric Pipeline

Threats to a biometric system exist at every stage of the pipeline built in this lab. At the capture stage, the primary threats are presentation attacks (spoofing) and sensor tampering. A face recognition system can be attacked by presenting a photograph, a video replay, or a 3D-printed mask to the camera sensor instead of a live face. Countermeasures include liveness detection algorithms that check for blink patterns, micro-expressions, or depth cues, and physical security controls over the sensor hardware itself. In this lab's context, the ORL dataset contains static photographs with no liveness information, meaning the capture stage has no anti-spoofing capability by design.

At the transmission stage, threats include man-in-the-middle interception of biometric data in transit between the capture sensor and the processing system, and replay attacks where a previously captured transmission is re-injected. Countermeasures are standard transport security measures: TLS encryption for data in transit, mutual authentication between sensor and server, and nonce-based or timestamped challenge-response protocols to defeat replay attempts.

At the storage stage, the threat model is database breach: an attacker who gains access to the stored template database extracts all enrolled biometric references simultaneously. Unlike a password database where individual entries can be cracked one by one, a biometric template database gives an attacker all enrolled identities at once, and those identities cannot be changed. Countermeasures include encryption of the template store at rest, strict access control limiting which system components and personnel can read templates, secure hardware enclaves for template storage, and cancelable biometric transforms that store a transformed version of the template rather than the template itself.

At the matching stage, threats include hill-climbing attacks, where an adversary iteratively queries the matcher with modified probe images and uses returned similarity scores to converge on a sample that crosses the acceptance threshold, and threshold manipulation attacks where a compromised matcher is configured to accept at an artificially loosened threshold. Countermeasures include rate limiting and lockout policies on failed match

attempts, score output suppression returning only accept/reject rather than the raw distance, and integrity verification of the matcher's configuration and threshold settings.

8.3 Access Control Applied to the Biometric Dataset

In this lab, access control was applied by restricting the biometric dataset and template objects to in-memory Python variables within the local runtime environment. No templates were written to shared network locations, external databases, or cloud storage. In a production deployment, this translates to role-based access control (RBAC) on the template database, where only the matching engine process has read access to stored templates; enrolment agents have write-only access; and no human operator has direct read access to raw templates at rest. Audit logging of every template access event, successful and failed, provides a forensic trail for detecting unauthorised access attempts. Physical access controls over the hardware hosting the template database, including server room access restrictions and tamper-evident sealing of storage devices, form the outermost layer of this access control perimeter.

8.4 Encrypting Stored Biometric Templates

To demonstrate practical template protection, the enrolled PCA templates were encrypted using Fernet symmetric encryption from the Python cryptography library. Fernet provides AES-128 encryption in CBC mode with PKCS7 padding and HMAC-SHA256 message authentication, ensuring both confidentiality and integrity of the stored template data.

```
from cryptography.fernet import Fernet
import numpy as np

key = Fernet.generate_key()
cipher = Fernet(key)

template_bytes = enroll_templates[0].tobytes()
encrypted = cipher.encrypt(template_bytes)

decrypted_bytes = cipher.decrypt(encrypted)
recovered = np.frombuffer(decrypted_bytes, dtype=enroll_templates[0].dtype)

print(f"Original (first 5): {enroll_templates[0][:5]}")
print(f"Recovered (first 5): {recovered[:5]}")
print(f"Round-trip verified: {np.allclose(enroll_templates[0], recovered)}")
print(f"Raw template size: {len(template_bytes)} bytes")
print(f"Encrypted template size: {len(encrypted)} bytes")
```

The round-trip encryption and decryption confirms that a template can be stored in encrypted form and recovered identically for matching with no loss of information. The encrypted form is opaque to anyone without the decryption key; it reveals neither the subject's identity nor any reconstructible approximation of their face. In production, the encryption key would be stored separately from the template database, ideally in a hardware security module (HSM) or dedicated secrets management system such as HashiCorp Vault, so that physical access to the template store alone is insufficient to decrypt the contents.

8.5 Cancelable Biometrics and Template Revocation

A fundamental limitation of the PCA templates generated in this lab is that they are linked to a fixed, unchangeable physical trait. If a template is compromised, the subject's face cannot be changed to invalidate the stolen template. Cancelable biometrics address this by applying

a non-invertible mathematical transformation to the raw template before storage, controlled by a revocable parameter unique to each deployment. If a cancelable template is compromised, the administrator revokes the transformation parameter and re-enrols the subject under a new one, producing a completely different stored template from the same underlying biometric. The compromised template becomes useless; the subject's genuine biometric is still available for fresh enrolment under the new parameter.

Standard PCA templates do not satisfy the non-invertibility requirement: as established in Task 3, PCA is a linear and approximately invertible transform, meaning an attacker with access to both the template and the eigenface basis vectors could reconstruct an approximation of the original face. Cancelable biometrics close this gap. Common schemes include BioHashing, which projects templates onto a random subspace seeded by a user-specific token, and salted template transforms analogous to password salting applied to feature vectors. Each scheme trades some matching accuracy for the security benefit of revocability, and the choice depends on the sensitivity of the application and the acceptable performance cost.

8.6 Privacy, Legal, and Ethical Implications

Biometric data is classified as sensitive personal data under most modern data protection frameworks. Nigeria's Data Protection Act 2023 (NDPA) explicitly includes biometric data in its definition of sensitive personal data, imposing heightened obligations on any organisation that processes it: a lawful basis for processing must be established, data subjects must be informed of how their data is used, retention periods must be defined and enforced, and cross-border transfers are subject to additional restrictions. The Nigeria Data Protection Commission (NDPC) has enforcement authority over violations. A biometric system deployed in Nigeria without NDPA compliance exposes the operating organisation to regulatory sanction, reputational damage, and civil liability to affected data subjects.

Beyond legal compliance, three ethical dimensions merit attention. The first is consent and power asymmetry: biometric enrolment in employment, border control, or law enforcement contexts is frequently not genuinely voluntary, as refusal may result in denial of services. Where consent is cited as the legal basis for processing, the voluntariness of that consent deserves scrutiny. The second is algorithmic bias: face recognition systems have been extensively documented to exhibit differential error rates across demographic groups, with significantly higher false acceptance and false rejection rates for darker-skinned individuals and women in systems trained predominantly on lighter-skinned male faces. The ORL dataset used in this lab is itself not demographically representative. Deploying a system without auditing error rates across demographic subgroups risks systematically disadvantaging certain populations at the point of authentication. The third is function creep: biometric data collected for one purpose, such as employee attendance tracking, is frequently repurposed for others, such as location monitoring or behavioural analysis, without the data subject's knowledge or additional consent. Technical controls alone cannot prevent function creep; governance policies, data minimisation requirements, and independent audit mechanisms are necessary complements to encryption and access control.

The permanent nature of biometric data makes these considerations more consequential than they are for conventional credentials. A person whose password is used without their knowledge can change it. A person whose face template is stolen, misused, or incorporated into a surveillance system without consent has no equivalent remedy. This asymmetry between a data subject's exposure and their available recourse is the core ethical challenge

of biometric system deployment, and it places a proportionally greater burden of care on the organisations that design, operate, and secure these systems.